

Real-time Notifications, Presence, and Collaboration — Research Index

Research session generated 2026-05-07. Each chapter is a standalone markdown note designed for Trilium ingest. Mermaid blocks render natively in Trilium markdown notes.

Chapters

1. Context — why this work, goals, HIPAA stance
2. Current-state audit — what exists today (verified), with file:line evidence
3. Target architecture overview — single-picture design + core abstraction
4. Event taxonomy & channel matrix — bare-minimum vs opt-in, audience, channels
5. Channel selection rules — decision tree, dedup, quiet hours
6. Sync vs async — when to fan out in-request vs via outbox vs scheduled
7. Realtime transport comparison — Ably / Pusher / Soketi / Centrifugo / Vercel-native
8. Presence model & channels — naming, auth, end-to-end sequence
9. Collision mitigation — optimistic locking, atomic signature, soft leases, CRDT
10. Phased rollout — gantt + per-phase deliverables
11. HIPAA & audit posture — PHI policy, BAAs, audit log
12. Files & modules to add — concrete drizzle/lib map
13. Open decisions — what to defer, with safe defaults
14. Verification plan — per-phase end-to-end checks
15. Recommendations summary — one-page distilled call

Reading order

If short on time: **Context → Architecture overview → Phased rollout → Recommendations summary** is the 4-chapter critical path. The rest is depth on demand.

Source

Approved plan archived at `~/claude/plans/give-our-current-stack-polished-dragon.md`.

Chapters

1. Context
2. Current State
3. Architecture Overview
4. Event Taxonomy
5. Channel Rules
6. Sync Vs Async
7. Realtime Comparison
8. Presence And Channels
9. Collision Mitigation
10. Rollout Phases
11. Hipaa And Audit
12. Files Modules
13. Open Decisions
14. Verification
15. Recommendations

00 — Context

The platform currently has zero real-time infrastructure. All user-facing async signal happens over Resend email, fired synchronously from inside route handlers. There are no push notifications, no SMS, no WebSocket/SSE, no background queue, and no general-purpose audit log. As the three-role model (Patient / PDA / Zabaca Agent) matures, the lack of a real-time signal path is becoming a UX bottleneck:

- A PDA uploads a record; the patient has no idea until they refresh.
- A release is created and needs a patient signature; only an email goes out, and the patient may not be reading email.
- An agent and a PDA can both touch the same release simultaneously and the last write wins silently.

Goals, in priority order

1. **Event → notification path** — a single domain-event abstraction that decides *who* gets notified, on *which* channel, *now or later*, with dedup.
2. **Presence** — know which users are currently online and on which screen, so we can downgrade push/email when in-app delivery is sufficient.
3. **Collision mitigation** — prevent silent data loss when two actors edit the same record.

HIPAA stance

Notification payloads omit PHI. "New record uploaded — open the app to view." Always. Anything that *could* contain PHI rides authenticated, BAA-covered transport (Resend, hosted realtime via BAA tier, or self-hosted in our VPC). This stance makes vendor choice a perf/cost question rather than a compliance question.

Roles glossary

- **Patient** — primary account holder, owns their data
- **PDA (Patient Designated Agent)** — surrogate decision-maker invited by the patient with per-relationship permissions (`healthRecordsPermission`, `manageProvidersPermission`, `releasePermission` $\in$ {viewer, editor})
- **Zabaca Agent** — internal staff role, can act on behalf of patients via `zabacaAgentRoles`

01 — Current-state audit

Verified by reading the codebase 2026-05-07.

Infrastructure inventory

Concern	Status	Evidence
Email	Active (Resend)	<code>apps/web/src/lib/email.ts:1-41</code> ; 9 templates already wired
SMS	None	—
Mobile push	None (mock only)	<code>apps/mobile/src/mock/notifications.ts</code> ; no <code>expo-notifications</code> import
Web push / Service Worker	None	—
In-app toast	Mantine <code><Notifications /></code>	<code>apps/web/src/app/layout.tsx:25</code>
In-app inbox	None	—
WebSocket / SSE / Pusher / Ably	None	—
Polling (refetchInterval)	None	—
Background jobs / Vercel Cron	None	<code>vercel.json</code> has no cron entries
<code>waitUntil</code> usage	None	—
Audit log (general HIPAA action log)	Missing	Only domain-specific: <code>fileUploadLog</code> , <code>releaseRequestLog</code> , <code>Session</code> (devices)
Optimistic locking	None anywhere	All updates are blind overwrites
Release-signing race	Open bug	<code>apps/web/src/app/api/releases/[id]/sign/route.ts:21-37</code> — check-then-update is non-atomic

Roles model (verified)

- `users.type ∈ {admin, user}` — coarse axis only
- Patient: any user with a `patientAssignments` row (assigned Zabaca agent)
- PDA: user with a row in `patientDesignatedAgents` , per-relationship scopes: `healthRecordsPermission` , `manageProvidersPermission` , `releasePermission ∈ {viewer, editor}`
- Zabaca Agent: row in `zabacaAgentRoles` (staff)
- Session payload (`apps/web/src/auth.ts:42-53`): `{ id, email, type, isAgent, isPda, isPatient, mustChangePassword, onboarded, disabled }`
- Mobile session: same JWT in `expo-secure-store` , decoded client-side (`apps/mobile/src/lib/api.ts:82-92`)

Existing email-trigger surfaces

These are the natural hook points for the new event publisher.

Event	Route handler	Current side-effect
Release created	apps/web/src/app/api/releases/route.ts:45-153	sendNewReleaseNotificationEmail to agent
Record uploaded by PDA/Agent for patient	apps/web/src/app/api/records/upload/route.ts:9-90	sendNewRecordUploadEmail to patient
PDA invite created	apps/web/src/app/api/my-designated-agents/route.ts:63-130	sendInviteEmail
Scheduled call created	apps/web/src/app/api/scheduled-calls/route.ts:43+	sendScheduledCallEmail
Release signed	apps/web/src/app/api/releases/[id]/sign/route.ts:9-38	No notification — gap
PDA invite accepted/declined	apps/web/src/app/api/representing/pending/[id]/route.ts:10-47	No notification — gap
Profile updated	apps/web/src/app/api/profile/route.ts:41-72	No audit, no notification — gap

Concurrency findings

All update endpoints reviewed perform blind overwrites — no `version` check, no `If-Match`, no `SELECT ... FOR UPDATE`, no lock tables.

Highest-risk surfaces:

- **Release form ↔ signing:** PUT release deletes/reinserts providers blindly (`apps/web/src/app/api/releases/[id]/route.ts:46-50`); concurrent edit from patient and PDA can interleave.
- **Signature:** check-then-update is non-atomic in `apps/web/src/app/api/releases/[id]/sign/route.ts:21-37`. Two simultaneous signs both succeed.
- **Health record metadata:** PATCH at `apps/web/src/app/api/documents/[id]/route.ts:89-90` has no prior-state check.
- **Profile:** PATCH/PUT at `apps/web/src/app/api/profile/route.ts:49-52, 65-66` — no version, no audit.

Form/draft persistence

None. Release form lives in React state only (`apps/web/src/components/release-form/ReleaseForm.tsx:34-55`); unsubmitted data is lost on navigation. No `localStorage` / `AsyncStorage` drafts.

02 — Target architecture overview

The plain-English version: think of it as a hotel

Imagine the platform is a hotel. Every guest (user) has a room (their account). Things happen during the day: room service (a record uploaded), a delivery for guest 204 (a release waiting for signature), a guest's friend showing up to be added to the room key (a PDA invite).

Today, every time something happens, the front desk clerk *personally* picks up the phone, calls the guest's room, AND mails them a letter, AND walks down the hall to slip a note under their door — all before they can serve the next person at the front desk. That's slow, the clerk is doing too many jobs, and they have no record of who did what.

The new design splits the work:

Hotel role	What it does	In our system
Front desk clerk	Takes the request, does the actual business work, writes one slip and drops it in the bell desk's tray	Route handler — calls <code>publishEvent(...)</code> once and returns to the user
Logbook	Records who came in, what they wanted, when, signed by the clerk	<code>audit_log</code> table — append-only history for HIPAA
Bell desk tray	Slips waiting to go out	<code>outbox</code> table in the database
Bellhop	Picks up slips and figures out <i>how</i> to reach each guest	Background drain (<code>waitUntil</code>) + the channel router
Receptionist's whiteboard	Tracks which guests are currently in the lobby vs. their room vs. out	Presence ledger (online / recently-online / offline)
Phone in the room	Reaches a guest who's in their room right now	In-app live notification (toast, inbox)
Pager / lobby intercom	Buzzes a guest who's somewhere on property	Push notification (mobile + web)
Mailbox in the lobby	Reaches a guest even after they've checked out for the day	Email
Hotel-wide PA	Realtime channel that broadcasts ID-only pings to everyone subscribed	Ably / Soketi

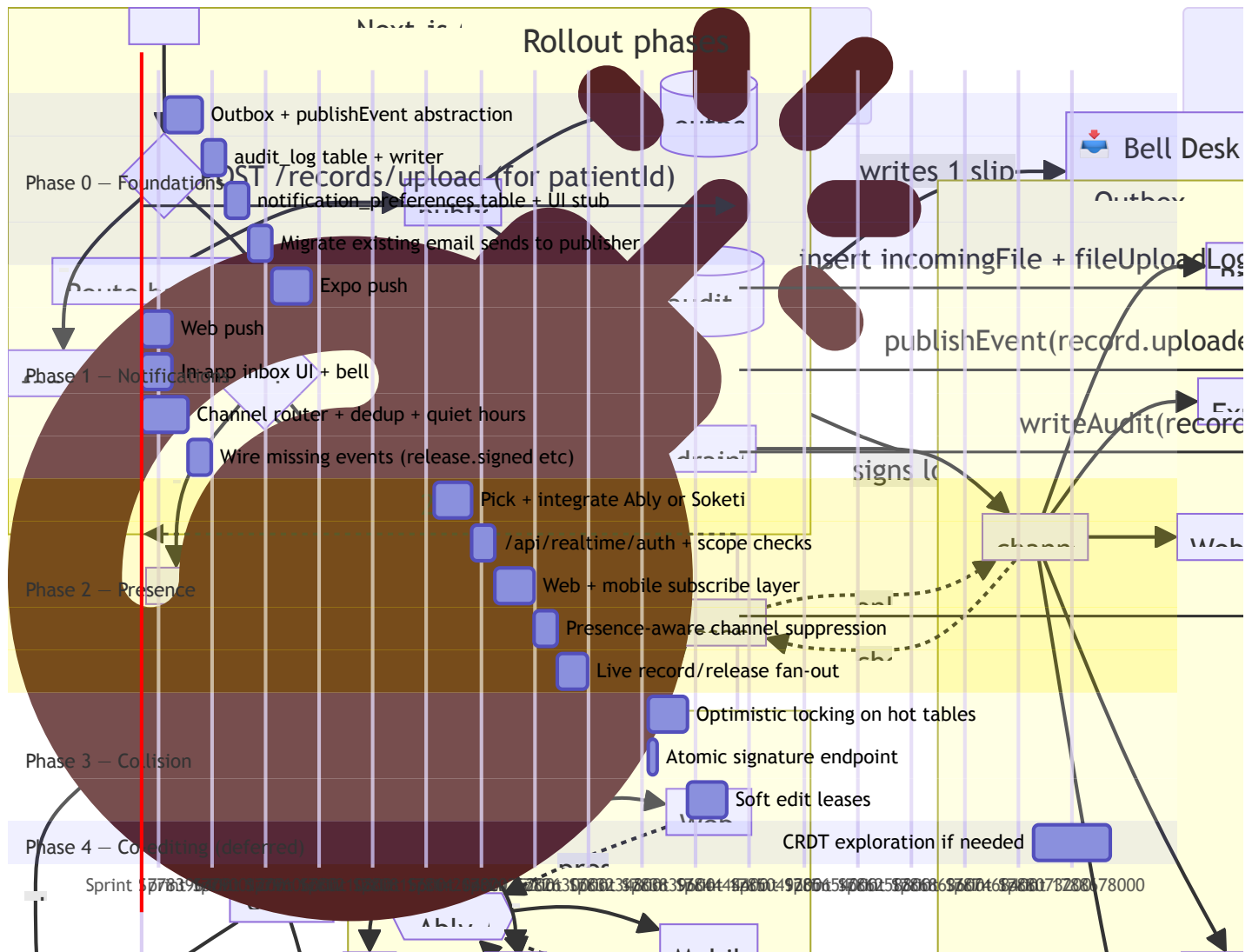
The bellhop's rule of thumb:

- "Is the guest in their room? Just ring the in-room phone." (in-app live)
- "Stepped out a few minutes ago? Page them." (push)
- "Gone for the night? Drop a letter in the mailbox." (email)
- "Their wallet's on fire? Do all three." (critical events)

And before any of that: "Did I already tell them this 30 seconds ago? Skip." (dedup)

That's the entire mental model.

Layman's-eye view of the flow



Things to notice:

1. **The front desk clerk is fast.** They do one thing — write a slip — then they're free to help the next guest. The guest doesn't wait for the bellhop to finish his rounds.
2. **The slip is durable.** Even if the bellhop has a bad day (server crash), the slip is still in the tray. The next bellhop on shift picks it up.
3. **The brain is in one place.** All the rules — "in-app vs. push vs. email", "have I already said this?", "is it quiet hours?" — live with the bellhop, not at the front desk.
4. **The logbook is separate from the slip.** The slip might get lost or duplicated; the logbook is permanent and never edited. Auditors talk to the logbook.

The technical diagram

Same picture, with the actual moving parts:

A worked example, end to end

Scenario: a PDA uploads a new health record for the patient they represent.

Read it as a story: PDA hands over a record, the clerk saves it and drops a slip, the clerk says "done" — *that's the user's whole experience*. Everything after is the bellhop on his rounds, invisible to the PDA, and shaped by where the patient happens to be at that exact moment.

Why this shape (and why not just call email directly like we do today)

Today	After
The route handler knows about Resend, has to import the email lib, has to pick the right template.	Route handler calls one function and forgets.
If we add push, the route handler has to learn about Expo too. And FCM. And VAPID.	Push is added once, in the bellhop. Every existing route handler benefits with no changes.
If a notification fails (Resend down for a minute), the user's request fails too.	The slip is in the tray; the bellhop retries. The user already got their 200.
No record of "we told user X about Y at time Z".	Every slip is logged; every send is logged.
Sending duplicate emails is just bad code waiting to happen.	The bellhop checks the dedup ledger before every send.
There's no place to add "is the user online right now?" logic.	The bellhop already asks. Adding presence-aware suppression is one config flag per event.

The four boundaries that keep this clean

1. **Route handlers never call channel adapters directly.** Always go through `publishEvent`. (One choke point for instrumentation, audit, and future changes.)
2. **Channel adapters never read the database for business data.** They get exactly the payload the bellhop hands them. (Keeps PHI out of vendor SDKs entirely.)
3. **The audit log is append-only and write-from-publisher-only.** Nothing edits it. Nothing else writes to it. (Trustworthy compliance evidence.)
4. **The realtime payload is always ID-only.** "Record 47892 changed — go look." Never "here are the lab results: ...". (Keeps PHI off the wire regardless of vendor BAA status.)

These four rules are what make the rest of the system safe to evolve. As long as we don't violate them, swapping email providers, adding SMS, switching from Ably to Soketi, or piping the audit log into Datadog all become local changes.

03 — Event taxonomy & channel matrix

The "mandatory" test

An event earns the top tier only if **at least one** of these is true:

1. The user **cannot perform a required action** without it (the notification IS the workflow — e.g., a signature link)
2. **Account access changes** (they can't log in or their access has been revoked)
3. **Security or trust boundary** event (someone else just got into their account)

Everything else — even important things — is "default-on but the user can mute the channel they don't want." The whole point of the floor is to keep it small enough that users don't fight us by ignoring all notifications.

Tier 1 — Mandatory (cannot be muted)

Four events. They fan out to every enabled channel and bypass quiet hours.

Event	Audience	Channels	Sync?	Why mandatory
release.signature_required	Patient	in-app + push + email	sync	The user cannot complete the workflow without seeing this. The email contains the signing link.
pda.invite_received	Invitee	email	sync	The invitee may not have an account yet; email IS the only path.
account.suspended / account.reinstated	User	email	sync	They can't log in to read an in-app message.
security.new_device_login	User	in-app + push + email	sync	Account-takeover signal; must reach them outside the app.

These four are the entire mandatory set. Everything else — including things that *feel* mandatory like "a record was just uploaded for me" — is in tier 2.

Tier 2 — Default-on, channel-tunable

Delivered by default; the user can mute *per channel* (e.g., turn off email but keep push), but cannot turn the event off entirely.

These are the events most users actually want; we just stop arguing about whether email-vs-push is the right channel for them.

Event	Audience	Default channels	Sync?
release.signed	Agent, PDA (viewer+)	in-app + push	async
release.declined	Agent, PDA (creator)	in-app + email	async
record.uploaded (when uploaded by PDA / Agent for patient)	Patient	in-app + push, email digest	async
pda.invite_accepted	Patient	in-app + push	async
pda.invite_declined	Patient	in-app + email	async
pda.permissions_changed	PDA	in-app + push + email	sync
scheduled_call.created / .reminder	Patient	in-app + push + email	async

pda.permissions_changed is borderline tier 1 (trust boundary), but it's not access-blocking — kept here so users can mute the email if they're in the app constantly.

Tier 3 — Opt-in (default OFF)

Power-user features. Off by default; surfaced in the preferences UI for users who want them.

- `record.viewed_by_pda` — see who's reading what
- `release.viewed` — recipient opened
- `profile.updated_by_pda` — when PDA edits patient profile
- `provider.added_by_pda` / `provider.removed_by_pda`
- `record.shared_with_pda` (audit-trail style notifications)
- Daily / weekly digest summaries (alternative to per-event emails)
- Marketing / product updates

Comparison to the previous draft

The earlier draft listed 11 mandatory events. The new floor is 4. Events that moved out of mandatory:

- `release.signed` / `release.declined` — informational ack; missing it doesn't break a workflow.
- `record.uploaded` — important but not blocking; users can opt to mute push if they upload often via PDA.
- `pda.invite_accepted` / `.declined` — courtesy update.
- `pda.permissions_changed` — kept default-on, but mutable; users who manage many PDAs may want to disable email noise.
- `scheduled_call.created` — calendar entry is the durable artifact; the notification is a reminder, not the workflow itself.

Audience derivation

Audience is computed at publish time from the event subject:

- For events on a `patientId` : include the patient + every PDA whose row on `patientDesignatedAgents` grants the relevant permission + every Zabaca agent assigned to the patient (filtered by event type).
- For events on a `userId` : just that user.
- For broadcast events (e.g., `account.suspended` to staff dashboards): expand via `zabacaAgentRoles` .

Audience computation lives in the publisher, not in route handlers, so role rules are centralized and testable.

What "mandatory" actually means at the channel level

Even tier-1 events respect *channel availability*. If the user has no push token registered, we don't try to send a push — we don't fail the event, we just skip that channel. "Mandatory" means the *event* cannot be muted; it does not mean every channel always fires.

Quiet hours are bypassed for tier 1 only. Tier 2 events scheduled during quiet hours are deferred to the next allowed window via the `outbox notBefore` field.

04 — Channel selection rules

Dedup

Each event carries a `dedupKey` (e.g., `record.uploaded:{recordId}:{audienceUserId}`) and a `cooldownSeconds` . The processor checks an in-Postgres `notification_log` (or Redis) before delivery.

Channel	Default cooldown
in-app	0s (always show — feed UX)
push	30s
email	5 min

Quiet hours

Per-user `quietHoursStart` / `quietHoursEnd` . Non-critical pushes get `notBefore` set to the next allowed window via the outbox. Critical events bypass quiet hours.

"Critical" definition

An event is critical if it requires user action within minutes (signature), affects account access (suspension), or is security-sensitive (new device login). Critical events fan out to every enabled channel regardless of presence — the user might be online on web but the alert is more important than minimizing noise.

Per-event cooldown overrides

Stored in the event definition (not in user prefs). Example: `pda.invite_received` has cooldown 0 because each invite is unique; `record.uploaded` for a bulk upload of 20 files coalesces to a single "20 new records" notification through a 60-second cooldown bucket.

Storage: what hits the database

Yes — running this rule engine means three new tables. Two are tiny by design.

`notification_preferences` — store overrides only

The naive approach is one row per `(user × eventType × channel)` . With ~20 events × 4 channels × N users that grows fast and is mostly noise (most users keep defaults).

The lean approach: **defaults live in code; the table stores only deviations.**

```
notification_preferences:
  user_id      - FK to users
  event_type   - string, e.g. "record.uploaded"
  channel      - enum: in_app | push | email | sms
  enabled      - bool
  updated_at
  PRIMARY KEY (user_id, event_type, channel)
```

A row exists only if the user explicitly toggled something off (or back on). When the router needs to decide, it reads any matching rows for that user once per event delivery and overlays them on the code-defined defaults. Cache per-user prefs in Redis or in-memory with a 60s TTL to skip the DB hit on bursty events.

Per-user settings that don't fit the matrix go on the user row itself (or a sibling 1:1 table):

```
users:
  ...
  quiet_hours_start - local time, nullable
  quiet_hours_end   - local time, nullable
  timezone          - IANA, e.g. "America/Los_Angeles"
```

How `timezone` gets populated

Device-reported, with optional user override. Not IP-geolocated.

- **Web:** `Intl.DateTimeFormat().resolvedOptions().timeZone` — every modern browser returns the IANA zone the OS is set to. Free, accurate, no permission prompt.
- **Mobile:** `expo-localization`'s `Localization.timezone` — same idea on iOS/Android.

The client sends this on every login and on session refresh. Server-side, the auth layer updates `users.timezone` if it changed. The most-recent device wins, which matches user expectation (a patient who flies LA → NYC for a week will start getting their quiet hours in Eastern time within one app open).

User override: a "Timezone" dropdown in the profile/settings page lets the user pin a value. If `users.timezone_locked = true`, the auto-update is skipped. This handles the "I live in Boston but I'm a snowbird in Florida every winter, just keep my quiet hours on Eastern" case.

Why not IP geolocation: we already capture lat/lng per session for the device-list UI, but IP-based zones lie regularly (VPNs, mobile carriers routing through different states, corporate proxies). Device-reported is the source of truth most users assume we use.

Why this matters less than it sounds: tier-1 events ignore quiet hours anyway, so timezone drift can only cause a tier-2 push to arrive at an inconvenient hour while traveling — annoying but not safety-relevant. Belt-and-suspenders: log the resolved zone on each notification send so we can audit if a user complains.

`notification_log` — dedup ledger + in-app inbox feed

This is the row written *every time* a notification is delivered (or suppressed). It powers two things at once: the dedup check and the user's "Inbox" UI.

```
notification_log:
  id
  user_id      - recipient
  event_type
  dedup_key    - indexed; the cooldown-window check reads this
  channel      - which channel fired (or null if suppressed)
  payload      - jsonb, no PHI; just IDs + display copy
  status       - sent | suppressed_dedup | suppressed_pref | suppressed_quiet | failed
  read_at      - for inbox UI; null until user opens
  created_at   - indexed
```

Indexes: `(user_id, created_at desc)` for the inbox; `(dedup_key, created_at)` for dedup lookups.

This is the single highest-write table in the system; budget accordingly. At healthcare-app scale (low-thousands of MAU) this is trivial Postgres. If/when it grows, partition by month or move dedup to Redis with a TTL keyed on `dedup_key`.

push_tokens — already covered in 11-files-modules

```
push_tokens:
  id
  user_id
  platform      - ios | android | web
  token         - Expo push token or VAPID endpoint+keys
  last_seen_at
  created_at
```

Where each rule reads from

Rule	Storage
"Is this event critical?"	Code (event definition)
"Does the user allow this channel for this event?"	<code>notification_preferences</code> overlaid on code defaults
"Is the user online right now?"	Realtime presence (Aby / Soketi / Redis); not in our DB
"Have we sent this dedup_key recently?"	<code>notification_log</code> (or Redis)
"Are we in the user's quiet hours?"	<code>users.quiet_hours_start/end</code> + <code>users.timezone</code>
"Does the user have a push token to send to?"	<code>push_tokens</code>

Two of those (presence, dedup) can move to Redis once volume warrants it; everything else stays in Postgres.

Migration shape

`notification_preferences` and `push_tokens` are added in **Phase 0** alongside the publisher and outbox. `notification_log` lands in **Phase 1** (it depends on the channel router being live). `quiet_hours_*` columns ship in Phase 1.

No backfill needed — empty preferences means "use defaults," which is the desired behavior for existing users.

05 — Sync vs async

Class	Path	Examples
Sync (in-request)	publish → outbox → fanout before response returns	<code>pda.invite_received</code> (email is the primary surface), <code>account.suspended</code> , <code>security.new_device_login</code>
Sync via in-process publish, async fanout	publish → outbox → <code>waitUntil(drain)</code>	Default for everything else
Scheduled	<code>notBefore</code> set on outbox row; cron drains overdue	digests, quiet-hours-deferred pushes, signature-required reminders

Recommendation

Start with `waitUntil` on the same Vercel Function. Migrate to **Inngest** or **QStash** only when:

- burst rate exceeds Vercel function timeout budget, or
- we need retries with exponential backoff and visible state.

Until then, the outbox table itself gives us a recovery surface — a separate cron-driven drain endpoint can re-process any rows stuck in `status = pending` past a retry threshold.

When sync is required

- The event drives the response copy ("we sent you an email") and the user expects it before page transition.
- The event is the authoritative side-effect (e.g., `pda.invite_received` 's email IS the invite — there's nothing else).
- Legal / security obligation to inform synchronously (`account.suspended` , `security.new_device_login`).

When async is the default

Everything else. The outbox commits in the same transaction as the business write, so the event is durable; the drain just routes it. If the drain fails, retry; the user already got their 200.

06 — Realtime transport comparison

Option	Type	HIPAA / BAA	Cost (est.)	DX	Pros	Cons
Abyl	Hosted	BAA on Enterprise tier	\$29–\$500+/mo	Excellent	Mature presence, message history, channel auth, SDKs for web+RN	Pricier; BAA only on top tier
Pusher Channels	Hosted	Limited (no BAA on most plans)	\$49+/mo	Good	Stable, well-known	BAA gap → only viable if PHI never touches payloads
PubNub	Hosted	BAA available	\$49+/mo	Good	HIPAA-eligible	Heavier API
Supabase Realtime	Hosted	BAA on Pro+	Bundled if on Supabase	Good	Free if already Supabase	We're not on Supabase — adopting it for one feature is a tax
Soketi (self-host)	Pusher protocol-compatible OSS	DIY (in-VPC)	\$5–\$20 VPS	Good	Pusher SDK ports if we ever switch hosted	Ops burden
Centrifugo (self-host)	OSS	DIY	\$5–\$20 VPS	Good	High-perf, presence built-in, JWT auth	Custom protocol; smaller community than Pusher
SSE via Vercel Functions	Native	DIY	Within Vercel	Limited	No new vendor	Function timeout cap (~5–15 min); breaks Fluid scaling; no presence; no fanout
Postgres LISTEN/NOTIFY → SSE	Native	DIY	Within Vercel	Limited	DB is the bus	Same Vercel function-lifetime problem; doesn't scale to thousands of concurrent listeners

Two paths to weigh

Hosted-first — Abyl

- Choose BAA tier from day one or stay on standard tier and strictly forbid PHI in any realtime payload (just IDs and event names — fetch details over authenticated REST).
- The "no PHI in transit" stance is already our default, which makes the standard tier viable.
- Fastest to ship; presence and history are turn-key.

Self-hosted-first — Soketi

- Pusher-protocol-compatible — `pusher-js` and `pusher-websocket-react-native` work unchanged, so a future swap to a hosted Pusher-protocol provider (e.g., Pusher itself, or Soketi's commercial peers) is trivial.
- Run in a small VPC (Fly.io machine, Render, or a \$5 Hetzner box). Add Redis for presence + horizontal scale.
- Slightly more setup, full HIPAA control, lower per-message cost at scale.

Vendor abstraction

Either way, all client code goes through a thin adapter:

- `apps/web/src/lib/realtime/client.ts` — server-side: publish + presence query
- `apps/mobile/src/lib/realtime.ts` — mobile subscribe layer

Switching vendors becomes a one-file change per surface.

What to skip

- **SSE via Vercel Functions** — Fluid Compute kills long-lived connections; doesn't fit the platform.
- **Postgres LISTEN/NOTIFY** — same lifetime issue, plus no scale beyond a few hundred listeners.
- **Pusher hosted** — BAA gap means we'd be locked into a "no PHI ever" stance with no upgrade path. Soketi gives us the same protocol with full control.

07 — Presence model & channels

Channel naming

<code>user:{userId}</code>	– private; user's own notifications
<code>patient:{patientId}</code>	– patient's data stream; subscribed by: – patient themselves – PDAs with any non-null permission – Zabaca agents assigned to patient
<code>release:{releaseId}</code>	– release detail page subscribers
<code>record:{recordId}</code>	– open-record viewers
<code>admin:broadcast</code>	– staff-only

Presence

- **Hosted (Ably):** use built-in presence on `patient:{id}` and `release:{id}` channels.
- **Self-host (Soketi):** use Pusher presence channels (`presence-patient-{id}`); Redis-backed.
- Server reads "is user X online?" via the realtime provider's REST API (Ably) or a Redis lookup we maintain (Soketi).

Channel auth

Every subscription request hits `POST /api/realtime/auth` :

1. Server validates the caller's session against the requested channel.
2. For `patient:{id}` : caller must be the patient, or a PDA with any permission row, or an assigned Zabaca agent.
3. For `release:{id}` : caller must be the patient, or a PDA with `releasePermission ≠ null` , or the agent who created it.
4. Returns a short-lived signed token (Ably tokens or Pusher signed-channel signature, ≤60s TTL).
5. Audit-log every issuance.

Sequence — record uploaded by PDA

What flows on the channel

Only IDs and event names. The client receives `{ type: "record.uploaded", recordId: "..."` } and fetches the record over the existing authenticated REST. This keeps PHI off the realtime wire entirely, regardless of vendor BAA status.

08 — Collision mitigation

Three layers, ordered by ROI.

8.1 Optimistic locking — phase 3a

Add a `version int not null default 1` column to: `users`, `releases`, `incomingFiles`, `userProviders`, `patientDesignatedAgents`.

Update pattern:

```
UPDATE releases SET ..., version = version + 1, updated_at = now()
WHERE id = $1 AND version = $2
RETURNING id;
```

If `RETURNING` is empty → 409 Conflict. Client refetches, shows merge UI, retries with new version.

All clients send the `version` they last loaded as part of any update body. Drizzle migration backfills `version = 1`.

8.2 Atomic signature — phase 3a (small, do alongside)

Replace `apps/web/src/app/api/releases/[id]/sign/route.ts:21-37` with a single SQL statement guarded by `WHERE auth_signature_image IS NULL`:

```
UPDATE releases SET auth_signature_image = $1, auth_date = $2, auth_expiration_date = $3
WHERE id = $4 AND auth_signature_image IS NULL
RETURNING id;
```

Empty `RETURNING` → 409 "already signed". Closes the existing race bug.

8.3 Soft edit leases — phase 3b

For UX-level "PDA Bob is editing this release":

- New table `record_locks { entityType, entityId, userId, leaseExpiresAt, sessionJti }` (or Redis with TTL).
- Client acquires a 30s lease on entering edit mode; heartbeats every 15s while editing; releases on close/blur.
- Other clients subscribing to the same `release:{id}` channel see lease events and render a "view-only — Bob is editing" badge.
- Lease expiry is enforced by the realtime channel + server-side check on save.
- This is **advisory**, not a true mutex. Optimistic locking remains the actual safety net.

8.4 CRDT — defer

Only consider Yjs / Automerge if a future feature introduces shared free-text editing (e.g., collaborative care notes). Liveblocks Yjs or `y-postgres` are the obvious paths. Out of scope for now — tracked as Phase 4.

Edit-mode flow

Summary

Optimistic locking + atomic signature endpoint = ~80% of the value for a few days of work. Soft leases polish the UX for the remaining 20%. CRDT is a phase-4 concern, only if a feature demands it.

09 — Phased rollout

Phase 0 — Foundations (no user-visible change)

- `outbox(id, eventType, payload jsonb, audience jsonb, dedupKey, cooldownSeconds, notBefore, status, attempts, createdAt, processedAt)` table
- `audit_log(id, actorUserId, actorRole, eventType, entityType, entityId, payloadDigest, ip, ua, createdAt)` — write-only, append-only
- `notification_preferences(userId, eventType, channel, enabled)` — defaults seeded
- `publishEvent(event)` API — single entry point, called from route handlers
- Refactor existing email call sites (releases POST, records upload, PDA invite, scheduled call) to use the publisher; the publisher's email channel still calls the same Resend templates → no behavior change

Exit criteria: every existing email flow now goes through `publishEvent`; `outbox` + `audit` rows are written; no user-visible change.

Phase 1 — Notifications complete

- `push_tokens(userId, platform, token, createdAt, lastSeenAt)` — Expo on mobile, VAPID on web
- Mobile: `expo-notifications` setup, permission flow, register-on-login, store token via `POST /api/me/push-tokens`
- Web: Service Worker + VAPID; same registration endpoint
- Channel adapters: `email.ts`, `expoPush.ts`, `webPush.ts`, `inApp.ts`
- Channel router: critical → multi-channel; otherwise prefs + dedup + quiet hours
- In-app inbox at `/inbox` (web) and `Inbox` tab (mobile); `notification_log` table feeds it
- Wire the missing events: `release.signed`, `pda.invite_accepted`, `pda.invite_declined`, `pda.permissions_changed`, `profile.updated_by_pda`
- **No SMS** (deferred per decision)

Exit criteria: a user with no in-app session still gets push for critical events. Preferences UI lets them mute non-critical channels.

Phase 2 — Presence + live updates

- Decision point: Ably (hosted) vs Soketi (self-host)
- `apps/web/src/lib/realtime/client.ts` and `apps/mobile/src/lib/realtime.ts` adapters; vendor concentrated in one module each
- `/api/realtime/auth` issues short-lived tokens after scope check; logs to `audit_log`
- Web: subscribe to `user:{me}`, `patient:{viewing}`, `release:{viewing}` based on route
- Mobile: same, plus background-suspend handling
- Channel router consults presence: skip push if user is on the relevant screen; downgrade email to digest if push delivered
- Outbox processor publishes to realtime as one of its fanout channels

Exit criteria: PDA uploads a record, patient sees it appear live without refresh; push is suppressed because patient is online.

Phase 3 — Collision mitigation

- Schema migration adds `version` column with default 1; backfill existing rows
- Update all hot-path mutations to require + check `version`
- Atomic signature endpoint refactor (1-line SQL change, deploy alongside)
- `record_locks` table or Redis; lease acquire/heartbeat/release endpoints; client edit-mode integration on releases first, profile + records second
- Conflict UI on 409: show diff, offer keep-mine / keep-theirs / merge

Exit criteria: two browser tabs cannot both sign the same release; concurrent edits surface a conflict UI rather than silently overwrite.

Phase 4 — Co-editing (only if a feature demands it)

- Yjs or Automerge for shared free-text fields; Liveblocks Yjs or self-hosted with `y-postgres`

10 — HIPAA, PII, and audit posture

Concern	Stance
Notification payloads	No PHI in titles/bodies/subjects. "New record uploaded — open the app to view." Always.
Realtime payloads	Only IDs + event names. Client fetches the actual record over authenticated REST/server actions.
Channel auth	Every subscribe request authenticated and scoped server-side; tokens short-lived (≤ 60 s); every issuance audit-logged.
BAAs	Resend (yes), Twilio (yes, when added), Ably (Enterprise only — confirm before paying), Expo Push (no PHI in payload anyway). Self-host avoids the BAA question entirely.
Audit log	Every event publish, every channel auth, every PDA-side data fetch on patient data, every signature attempt. Append-only <code>audit_log</code> .
Push token storage	<code>push_tokens</code> rows tied to <code>userId</code> ; revoke on logout, on session revocation, and on token rotation (Expo can rotate).
Quiet hours / opt-out	Per-user, per-channel, per-event-type. Hard-required-channels (signature, security, suspension) cannot be muted, only retargeted.

Audit log shape

```
audit_log:
  id
  actor_user_id    — who did the thing (or NULL for system events)
  actor_role       — patient | pda | agent | admin | system (denormalized for filtering)
  event_type       — namespaced string (record.uploaded, release.signed, channel.subscribed, ...)
  entity_type      — release | record | profile | invite | session
  entity_id        — the affected row's PK
  payload_digest   — sha256 of the event body (proves what was emitted without storing PHI)
  ip
  ua
  created_at       — indexed
```

Indexes: `(actor_user_id, created_at)`, `(entity_type, entity_id, created_at)`, `(event_type, created_at)`.

Retention: keep indefinitely (or per HIPAA's 6-year minimum, whichever is longer). Pipe to Datadog/Axiom later for query convenience; Postgres remains the system of record.

Why this stance simplifies vendor choice

Because PHI never leaves authenticated REST + the database, vendor BAA status is a defense-in-depth concern, not a compliance gate. Ably standard tier, Soketi self-hosted, Expo Push, and Resend are all viable. The decision becomes about cost, ops burden, and DX rather than legal risk.

11 — Files & modules to add

```
apps/web/src/lib/events/
  publisher.ts          // publishEvent(event)
  outbox.ts             // enqueue / drain / retry
  processor.ts          // fanout to channels
  router.ts             // channel selection rules + dedup + quiet hours
  types.ts              // event union – re-exported from packages/types

apps/web/src/lib/notifications/
  channels/email.ts     // wraps existing Resend templates
  channels/expoPush.ts  // calls Expo Push API
  channels/webPush.ts   // VAPID
  channels/inApp.ts     // writes notification_log row + realtime publish
  preferences.ts        // CRUD on notification_preferences

apps/web/src/lib/realtime/
  client.ts             // server-side: publish + presence query
  auth.ts               // /api/realtime/auth handler
  scope.ts              // role × entity authorization for channel subscribe

apps/web/src/lib/audit/
  log.ts                // writeAudit(...) – single entry point

apps/mobile/src/lib/
  push.ts               // expo-notifications register/handle
  realtime.ts           // subscribe layer

packages/types/src/schemas/
  events.ts             // EventName union, payload schemas
  notifications.ts      // preference shape, channel enum
```

DB migrations (drizzle)

New tables:

- `outbox`
- `audit_log`
- `notification_preferences`
- `notification_log` — for dedup + in-app inbox feed
- `push_tokens`
- `record_locks` (phase 3b — or use Redis)

Schema changes (phase 3a):

- ALTER `users`, `releases`, `incomingFiles`, `userProviders`, `patientDesignatedAgents` ADD COLUMN `version int not null default 1`

API routes to add

- `POST /api/me/push-tokens` — register/refresh token (mobile + web)
- `DELETE /api/me/push-tokens/:id` — revoke
- `GET /api/me/notifications` — inbox feed
- `POST /api/me/notifications/:id/read` — mark read
- `GET /api/me/notification-preferences` / `PUT` — preferences UI
- `POST /api/realtime/auth` — channel-subscribe token issuance (phase 2)
- `POST /api/locks/:entityType/:entityId` — acquire lease (phase 3b)
- `POST /api/locks/:entityType/:entityId/heartbeat`
- `DELETE /api/locks/:entityType/:entityId` — release

Existing routes to refactor

Replace direct email sends with `publishEvent` calls. Keep the email templates; the publisher's email channel still uses them.

Route	Current	After phase 0
<code>apps/web/src/app/api/releases/route.ts:137</code>	<code>sendNewReleaseNotificationEmail</code>	<code>publishEvent({ type: "release.created", releaseId })</code>
<code>apps/web/src/app/api/records/upload/route.ts:76</code>	<code>sendNewRecordUploadEmail</code>	<code>publishEvent({ type: "record.uploaded", recordId, byUserId })</code>
<code>apps/web/src/app/api/my-designated-agents/route.ts:116</code>	<code>sendInviteEmail</code>	<code>publishEvent({ type: "pda.invite_received", inviteId })</code> (sync)
<code>apps/web/src/app/api/scheduled-calls/route.ts:43+</code>	<code>sendScheduledCallEmail</code>	<code>publishEvent({ type: "scheduled_call.created", callId })</code>
<code>apps/web/src/app/api/releases/[id]/sign/route.ts</code>	(no notification)	<code>publishEvent({ type: "release.signed", releaseId })</code>
<code>apps/web/src/app/api/representing/pending/[id]/route.ts</code>	(no notification)	<code>publishEvent({ type: "pda.invite_accepted" / "pda.invite_declined" })</code>
<code>apps/web/src/app/api/profile/route.ts</code>	(no audit)	<code>writeAudit</code> only (no notification unless edited by PDA)

12 — Open decisions

Decision	Default if not chosen	Re-evaluate when
Ably vs Soketi	Defer to phase 2 entry; both modeled	Phase 2 kickoff
Inngest vs <code>waitUntil</code> -only	<code>waitUntil</code> first	Outbox backlog grows or retries get visible
In-app inbox: separate page vs notification drawer	Drawer first (faster), page later	Phase 1 design review
Audit log: separate Postgres table vs structured logs to Datadog/Axiom	Postgres table now; pipe to log aggregator later	When query needs exceed Postgres-friendly patterns
Push for web: which icon/badge UX	Match mobile patterns	Phase 1 design review
Conflict UI for 409	Phase 3 spike — start with "keep mine / keep theirs" only	First user-reported merge surprise
SMS (Twilio)	Deferred (per decision)	If a critical urgency path emerges that email + push can't cover
CRDT (Yjs/Automerger)	Deferred to phase 4	If a shared free-text editing feature is scoped

Decisions already made

- **Mobile push + web push** in phase 1
- **Defer SMS**
- **Present both Ably and Soketi** in research; pick at phase 2
- **HIPAA**: PHI never on notification or realtime payloads (regardless of vendor BAA)
- **Outbox in Postgres** as the durability surface

13 — Verification plan

End-to-end checks per phase. Local dev: `~/ .bun/bin/bun run dev .`

Phase 0

- Unit: `publishEvent` writes outbox + audit row in one transaction
- Existing email tests continue to pass after refactor
- `bun run type-check && bun run lint && bun run build`
- Spot-check: trigger PDA invite, see email still arrives + new outbox row + new audit_log row

Phase 1

- Mobile: register on a real device, send `release.signature_required`, observe push
- Web: install SW, subscribe, send same event, observe banner
- Dedup: trigger same event twice within cooldown — second suppressed
- Preferences: toggle off email for `record.uploaded`, trigger event, only push delivered
- Inbox: triggered events show up in `/inbox` (web) and Inbox tab (mobile)
- Quiet hours: set window, trigger non-critical event mid-window, push deferred to window edge

Phase 2

- Presence: open patient's portal in browser → mobile sends `record.uploaded` → web shows live toast, no push delivered
- Close web, wait 35s → trigger again → push delivered
- Channel auth: try subscribing to `patient:{otherPatientId}` as PDA without permission → 403, audit row written
- Token TTL: capture token, wait 90s, attempt subscribe → 401
- Two devices online for same user: both receive live events; neither receives a push

Phase 3

- Two browser tabs sign the same release simultaneously → exactly one 200, one 409
- Two tabs PUT same release with stale version → exactly one 200, one 409
- Two tabs enter edit mode → second sees "X is editing" badge; lease expires after 30s without heartbeat → second can take over
- Conflict UI: trigger 409, verify keep-mine / keep-theirs both work; verify retry uses fresh version

Cross-cutting

- Audit log: every test above produces the expected `audit_log` rows
- Mobile Metro resolution: `cd apps/mobile && node -e "require('./metro.config.js')"` after each phase
- Type check + lint clean at every phase boundary

14 — Recommendations summary

A one-page distilled view for stakeholders.

Realtime transport

Present both Ably and Soketi paths in the phase 2 entry doc; pick at the start of phase 2 based on traffic forecast and BAA needs. Keep all vendor code behind the `realtime/client.ts` adapter so the swap is mechanical.

Push

Mobile (Expo) + web (VAPID) in phase 1. Both register through a single `POST /api/me/push-tokens` endpoint.

SMS

Deferred. Revisit only if a BAA-justified urgent path emerges (e.g., signature reminders that must reach a phone outside the app).

Async

Outbox + `waitUntil` first. Inngest on demand — the outbox table itself is the recovery surface, and a cron-driven re-drainer covers retry until we need fancier orchestration.

Collision

Optimistic locking + atomic signature endpoint = ~80% of the value for a few days of work. Soft leases polish UX. CRDT only if a feature demands it.

HIPAA

Keep PHI off all notification and realtime payloads, regardless of vendor BAA status. This downgrades vendor choice from a compliance question to a perf/cost question.

Sequencing

Phase	What	Time
0	Outbox + publishEvent + audit_log; refactor existing emails through publisher	~2 weeks
1	Mobile push + web push + in-app inbox + dedup + missing event hooks	~3 weeks
2	Realtime + presence + presence-aware channel router	~3 weeks
3	Optimistic locking + atomic signature + soft leases	~2 weeks
4	CRDT (only if a feature demands it)	TBD

Single most impactful first move

Phase 0 alone closes 3 outstanding gaps:

1. Adds the general HIPAA audit log (`audit_log` table) the platform currently lacks
2. Centralizes notification logic so phase 1 work is purely additive
3. Surfaces missing event hooks (`release.signed` , `pda.invite_accepted` , etc.) without shipping new user-facing behavior

It is the lowest-risk, highest-leverage first step.